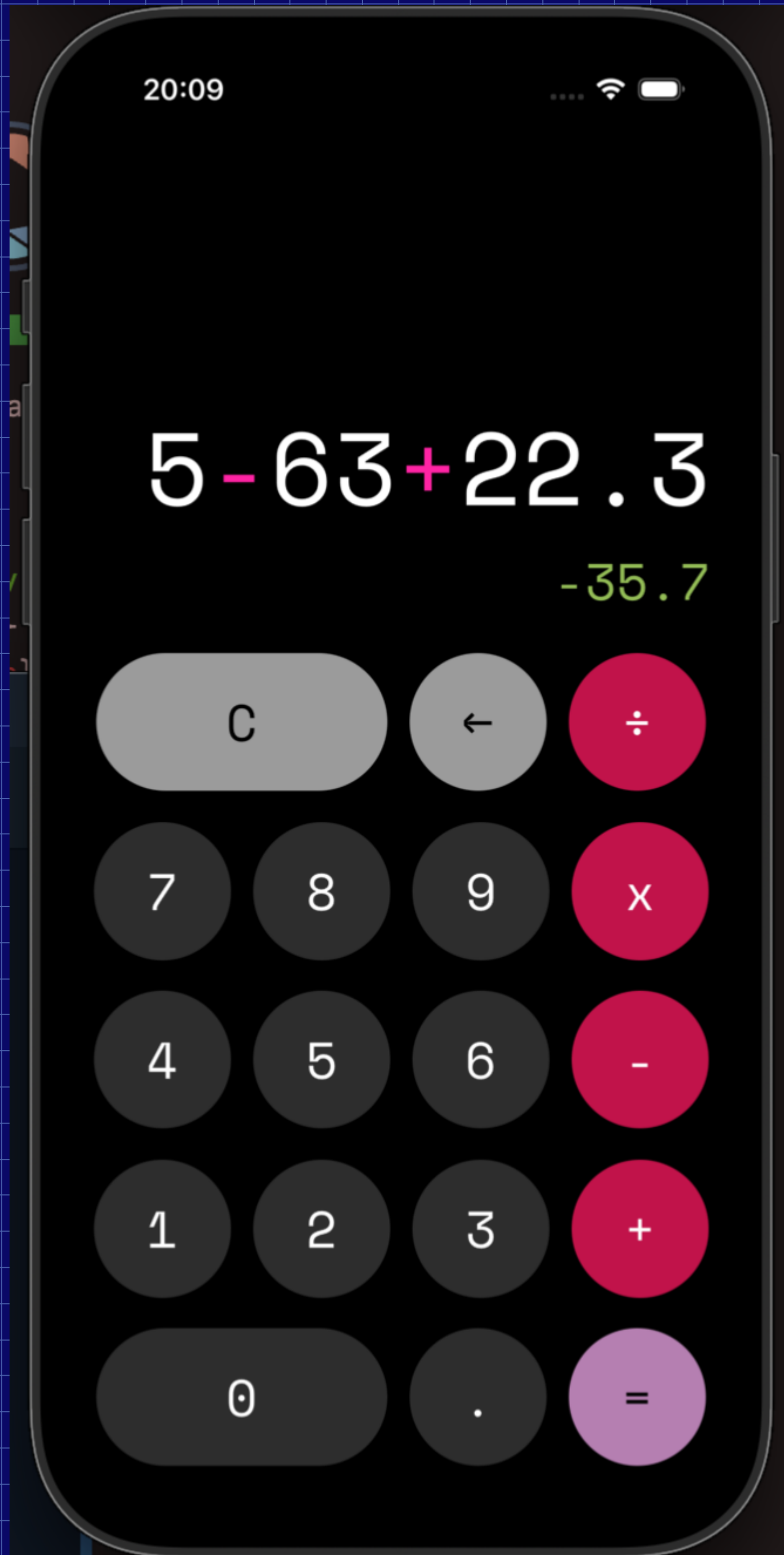


Calculator App

By Abril.

September '26.

The very first mobile app that I created.
Made with love ♥ and `React Native` with `Expo`.

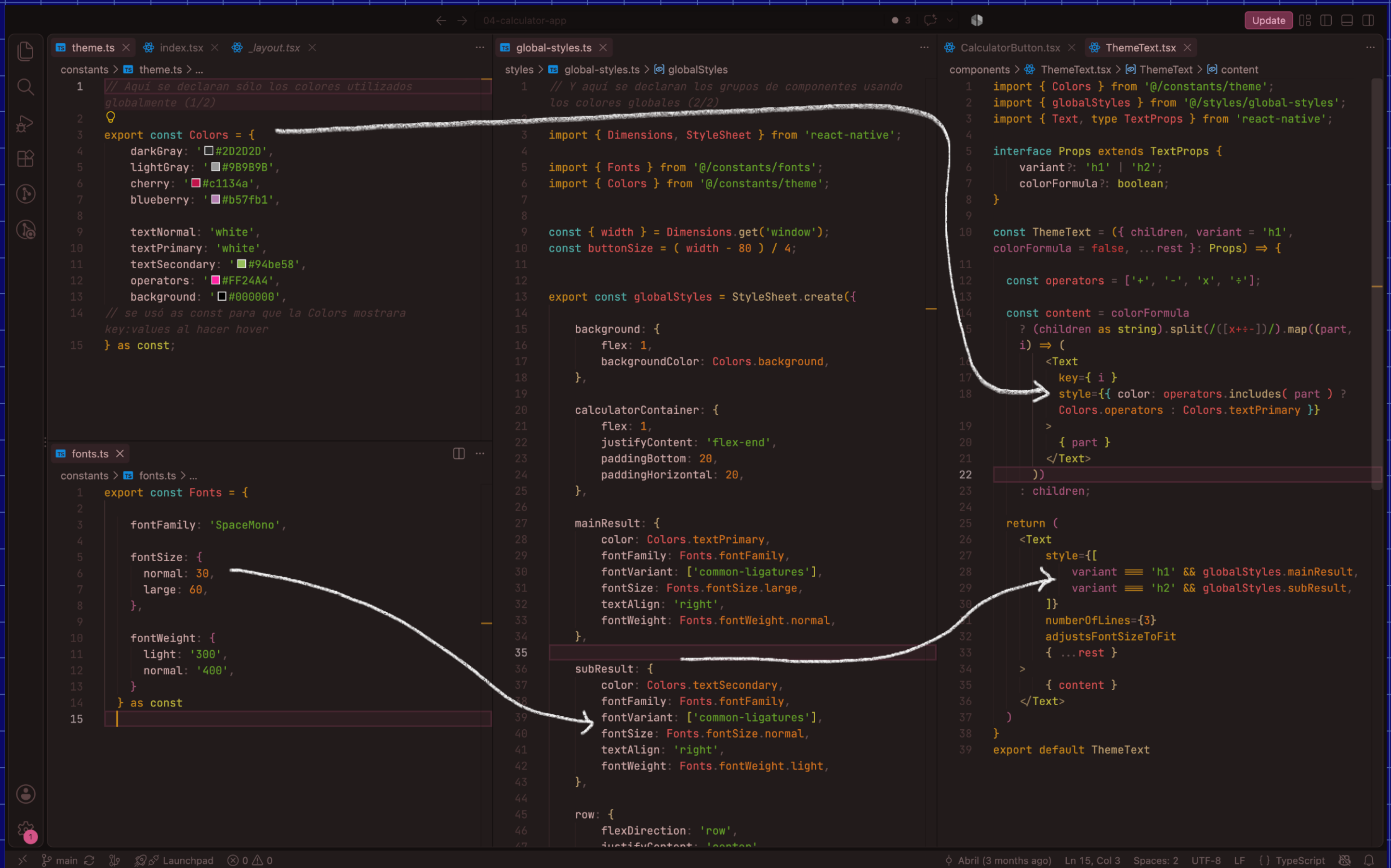


Dependencies

> Expo SDK
expo-haptics

> Third party
math.js

Styles



1 Global style variables are created.

- Color
- Fonts

2 Global style components are created w/ color n Fonts

- Create an s.c when it has +1 props

3 G.S.V and G.S.C are consumed.

Components

> Calculator Button

```
CalculatorButton.tsx X
components > CalculatorButton.tsx > CalculatorButton
1  import { Pressable, Text } from 'react-native';
2
3  import { Colors } from '@constants/theme';
4  import { globalStyles } from '@styles/global-styles';
5
6  import * as Haptics from 'expo-haptics';
7
8
9  interface Props {
10    label: string;
11    color?: 'lightGray' | 'darkGray' | 'cherry' | 'blueberry';
12    blackText?: boolean;
13    doubleSize?: boolean;
14    onPress: () => void;
15  }
16
17  const CalculatorButton = ({
18    label,
19    color = 'darkGray',
20    blackText = false,
21    doubleSize = false,
22    onPress
23  }: Props) => {
24
```

1 Interface for component properties are created and defined.

2 Interface Props is consumed and set to default values in some cases.

```
04-calculator-app - CalculatorButton.tsx
1  const colorStyles = {
2    lightGray: Colors.lightGray,
3    darkGray: Colors.darkGray,
4    cherry: Colors.cherry,
5    blueberry: Colors.blueberry
6  }
7
8  return (
9    <Pressable
10      style={({ pressed }) => ({
11        ...globalStyles.button,
12        backgroundColor: colorStyles[color],
13        opacity: pressed ? 0.5 : 1,
14        width: doubleSize ? 170 : 80,
15      })}
16      onPress={() => {
17        Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Heavy)
18        onPress()
19      }}
20    >
21      <Text
22        style={{
23          ...globalStyles.buttonText,
24          color: blackText ? 'black' : 'white',
25        }}
26      >
27        { label }
28      </Text>
29    </Pressable>
30  )
```

3 Color styles are defined w/ q.s.v

4 button from q.s.c is consumed but the background color can change.

5 There may be multiple onPress functs.

6 The same as 4

> Theme Text

```
04-calculator-app - ThemeText.tsx

1 interface Props extends TextProps {
2   variant?: 'h1' | 'h2';
3   children: string;
4 }
5
6 const ThemeText = ({ children = '', variant = 'h1', ...rest }: Props) => {
```

⊥ "children" is explicitly declared cause it needs to be modified within the component.

```
04-calculator-app - ThemeText.tsx

1 const operators: string[] = CALCULATOR_OPERATORS;
2 const formParts: string[] = children.split(/([x+÷-])/);
3
4 const content = variant === 'h1' ? *
5   formParts.map((part, i) => (
6     <Text
7       key={ i }
8       style={{ color: operators.includes( part ) ? Colors.operators : '' }}
9     >
10       { part }
11     </Text>
12   ))
13   : children;
```

⊥ '/' represents an regex.

'()' includes the operators within the array.

'[]' treats each character inside as a separate match rather than a whole string match.

≡ Goes to every item of an array and transforms each one with a provided function and returns a new array.

```
04-calculator-app - ThemeText.tsx

1 return (
2   <Text
3     style={{
4       variant === 'h1' && globalStyles.mainResult,
5       variant === 'h2' && globalStyles.subResult,
6     }}
7     numberOfLines={3}
8     adjustsFontSizeToFit
9     { ...rest }
10   >
11     { content }
12   </Text>
13 )
```

⊥ Allows to use the remain props when it's consumed.

≡ $\begin{matrix} <Text> \\ | \\ <Text> 1 </Text> \\ | \\ <Text> + <Text> \\ | \\ <Text> 3 <Text> \\ </Text> \end{matrix}$ *father* *children*

= $<Text> 1 + 3 </Text>$

Displays within the same line!

Hooks

> useCalculator

My logic: Use a single string for the entire formula and to perform the validations, extract the last numeric value by converting the string into an array using operators as delimiters, and use `Math.js` to calc the final formula string.

```
04-calculator-app - useCalculator.tsx

1
2 const [formula, setFormula] = useState('0');
3 const [result, setResult] = useState('0');
4
5
6 useEffect(() => {
7   if (formula !== '0') {
8     calculateResult();
9   }
10 }, [formula]);
11
12
13 const getLastNumber = (): string => {
14   // Separate each number (splited by operators) to get the last one
15   const parts = formula.split(OPERATOR_REGEX);
16   return parts[parts.length - 1];
17 }
18
19 const clean = () => {
20   setFormula('0');
21   setResult('0');
22 }
23
24 const deleteLast = () => {
25   if (formula.length === 1) {
26     setFormula('0');
27   } else {
28     setFormula(formula.slice(0, -1));
29   }
30 }
```

2 Reacts to 'formula' and calculates the result when 'formula' has new changes in real time.

3 Split the string into an array only of numeric values and extract the last value.
Is used in `buildFormula()`.

5 From index 0, except the last one.

Negative numbers represents positions counted from the end.

h	0	1	2
0	1	2	3
-3	-2	-1	

```
04-calculator-app - useCalculator.tsx

1 const newStart = () => {
2   setFormula(result.toString());
3   setResult('0');
4 }
```

6 I decided not to use the "=" button for calculate the result cause the result doesn't need to be manually pressed to display.
`useEffect(2)` does that automatically.
So instead I decided to use the "=" button to restart the result and replace the formula for it.

```

1  const buildFormula = ( newDigit: string ) => {
2    let lastNumber = getLastNumber();
3
4    // To avoid formula = operator ( except for "-" )
5    {
6      if( formula === '0' && ['x', '÷', '+'].includes( newDigit ) ) return;
7      if( formula === '-' && ['x', '÷', '+'].includes( newDigit ) ) return;
8
9      // To delete the first 0 when a new digit (not 0) is added, to have lastNumber = 5 instead = 05
10     if( lastNumber === '0' && newDigit !== '.' ){
11       setFormula(formula.slice(0, -1) + newDigit);
12       return;
13     }
14
15     // To put a 0 before a point when lastNumber is empty
16     if( lastNumber === '' && newDigit === '.' ){
17       return setFormula( formula + '0' + newDigit );
18     }
19
20     // To avoid having more than one period
21     if( lastNumber.includes('.') && newDigit === '.' ) return;
22
23     // To avoid lastNumber = 0000 when it's not 0.000
24     if( lastNumber === '0' && newDigit === '0') return;
25
26     // To replace the last sign for the new one instead of adding it to the formula
27     if( CALCULATOR_OPERATORS.includes(formula.slice(-1)) && CALCULATOR_OPERATORS.includes( newDigit ) ){
28       setFormula( formula.slice(0, -1) + newDigit );
29       return;
30     }
31
32     setFormula( formula + newDigit );
33   }

```

Formula = 5 + 10 x 3

① →

② → works for arrays and strings

③ {

④ {

⑤ {

⑥ {

⑦ {

⑧ {

⑨ {

⑩ Returns the last value

> To avoid:

f = ÷ → 3

f = 3 + x → 8

f = 05 → 4

f = 25 + . → 5

f = 2.72. → 6

f = 000 → 7



04-calculator-app - useCalculator.tsx

```
1  const calculateResult = () => {
2
3    // To get the qty of the numeric values
4    const parts = formula.split(OPERATOR_REGEX);
5    const nonEmptyParts = parts.filter(part => part !== '').length;
6
7    ① { if( nonEmptyParts < 2 ){ ②
8        setResult('0');
9        return;
10   }
11
12   let expr = formula;
13
14   ③ { if( CALCULATOR_OPERATORS.some(op => expr.endsWith(op)) ){
15       expr = expr.slice(0, -1);
16   }
17
18   const expression = expr
19     .replace(/x/g, '*')
20     .replace(/÷/g, '/'); ④
21
22   const res = evaluate(expression);
23   const resFixed = parseFloat(res.toFixed(5));
24   setResult( resFixed.toString() ); ⑤
25 }
```

1 Avoids having to calculate the result for an incomplete formula.

$f = 33 +$

2 Returns a new array that meets the conditions.

3 If the last string char is an operator, is removed.

5 Allows only 5 decimals.

Index

Calculator Button
is consumed in
index.

04-calculator-app - index.tsx

```
1 <CalculatorButton
2   label='C'
3   color='lightGray'
4   blackText={true}
5   doubleSize={true}
6   onPress={() => clean()} } haptics and
7 /> clean()
```

04-calculator-app - index.tsx

```
1 <CalculatorButton
2   label='C'
3   color='lightGray'
4   blackText={true}
5   doubleSize={true}
6   onPress={() => clean()}
7 />
```

04-calculator-app - index.tsx

```
1 <CalculatorButton
2   label='='
3   color='blueberry'
4   blackText={true}
5   onPress={() => newStart()}
6 />
```

04-calculator-app - index.tsx

```
1 <CalculatorButton
2   label='←'
3   color='lightGray'
4   blackText={true}
5   onPress={() => deleteLast()}
6 />
```

04-calculator-app - index.tsx

```
1 <View style={{ marginBottom: 20 }}>
2   <ThemeText variant='h1'>{formula}</ThemeText>
3   <ThemeText variant='h2'>{result}</ThemeText>
4 </View>
```